COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	08-28-2026	**DATE SUBMITTED:**	08-28-2026
NAME:	GUARIN, JAN MIKAELA C.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

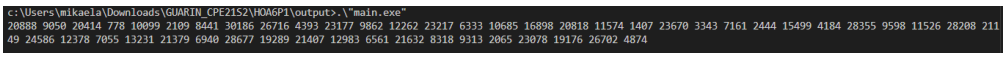
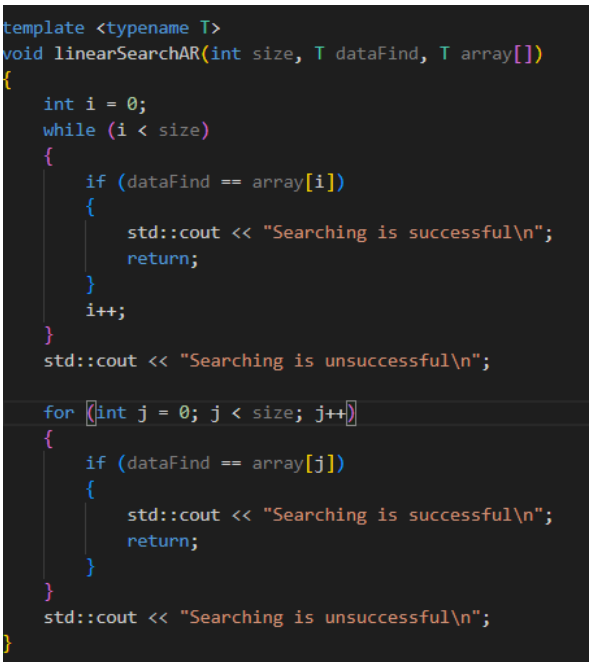
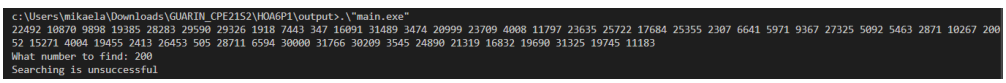
Screenshot	
Observations	The code creates an integer array with a maximum size of 50. It uses srand(time(0)) to make the random numbers different each time the program runs. The for loop fills the array with random numbers using rand(), then the numbers are displayed using std::cout. After that, the program asks the user to enter a number to find and passes the array and the chosen number to the linearSearch() function.

Table 6-1. Data Generated and Observations

Code	
Screenshot	

```
template <typename T>
void linearSearchAR(int size, T dataFind, T array[])
{
    int i = 0;
    while (i < size)
    {
        if (dataFind == array[i])
        {
            std::cout << "Searching is successful\n";
            return;
        }
        i++;
    }
    std::cout << "Searching is unsuccessful\n";

    for (int j = 0; j < size; j++)
    {
        if (dataFind == array[j])
        {
            std::cout << "Searching is successful\n";
            return;
        }
    }
    std::cout << "Searching is unsuccessful\n";
}
```

Observations	The code first generates and displays 50 random numbers. It then asks the user what number to find. The linearSearch() function receives the array, its size, and the number entered by the user. It checks the array elements one by one to look for the chosen number. In the screenshot, the number entered was 200, and the search was unsuccessful because 200 was not found in the generated array.

Table 6-2a. Linear Search for Arrays

Code

```
    // Creating a node for your name
    Node<char> *char1 = new_node('M');
    Node<char> *char2 = new_node('I');
    Node<char> *char3 = new_node('K');
    Node<char> *char4 = new_node('A');
    Node<char> *char5 = new_node('E');
    Node<char> *char6 = new_node('L');
    Node<char> *char7 = new_node('A');

    char1->next = char2;
    char2->next = char3;
    char3->next = char4;
    char4->next = char5;
    char5->next = char6;
    char6->next = char7;

    linearSearchLL(char1, 'K');

    return 0;
}
```

```cpp
template <typename T>
void linearSearchLL(Node<T> *nodes, T dataFind)
{
    while (nodes != nullptr)
    {
        if (nodes->data == dataFind)
        {
            std::cout << "The data: " << nodes->data << " was Found successfully.\n";
            return;
        }
        nodes = nodes->next;
    }
    std::cout << "Searching is unsuccessful.\n";
}

template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind)
{
    int low = 0, high = arrSize - 1, middle;
    while (low <= high)
    {
        middle = low + (high - low) / 2;
        if (dataFind == data[middle])
        {
            std::cout << "The data was found\n";
            return;
        }
        else if (dataFind < data[middle])
        {
            high = middle - 1;
        }
        else
        {
            low = middle + 1;
        }
    }

    std::cout << "The data was not found\n";
}
```

Screenshot

```
c:\Users\mikaela\Downloads\GUARIN_CPE21S2\HC
The data: K was Found successfully.
```

Observations

The code creates seven linked-list nodes containing the characters M, I, K, A, E, L, and A. Each node is connected to the next node using the next pointer, forming the linked list. The linearSearchLL() function then searches the linked list for 'K'. Since K is the third node in the list, the program finds it and displays “The data: K was found successfully.”

Table 6-2b. Linear Search for Linked List

Code

```
// generate a sorted array: (use for loop)
int newArr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for (int i = 0; i < max_size; i++)
{
    newArr[i] = i;
}

binarySearchArr(newArr, max_size, 100);

Node<char> *char1 = new_node('M');
Node<char> *char2 = new_node('I');
Node<char> *char3 = new_node('K');
Node<char> *char4 = new_node('A');
Node<char> *char5 = new_node('E');
Node<char> *char6 = new_node('L');
Node<char> *char7 = new_node('A');

// Connect the nodes
char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = char5;
char5->next = char6;
char6->next = char7;
```

```
template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind)
{
    int low = 0, high = arrSize - 1, middle;
    while (low <= high)
    {
        middle = low + (high - low) / 2;
        if (dataFind == data[middle])
        {
            std::cout << "The data was found\n";
            return;
        }
        else if (dataFind < data[middle])
        {
            high = middle - 1;
        }
        else
        {
            low = middle + 1;
        }
    }

    std::cout << "The data was not found\n";
}
```

Screenshot

```
The data was not found
```

Observations

The code first generates a sorted array from 1 to 10 using a for loop and searches for 100 using binarySearchArr(). Since 100 is not in the array, the search is unsuccessful. The code then creates a linked list containing M, I, K, A, E, L, A and connects each node to the next one. However, the shown output is “The data was not found”, because the value being searched for is not found by the binary search function in the linked list.

Table 6-3a. Binary Search for Arrays

Code

```
    // generate a sorted array: (use for loop)
    int newArr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    for (int i = 0; i < max_size; i++)
    {
        newArr[i] = i;
    }

    binarySearchArr(newArr, max_size, 100);

    Node<int> *char1 = new_node(10);
    Node<int> *char2 = new_node(11);
    Node<int> *char3 = new_node(12);
    Node<int> *char4 = new_node(13);
    Node<int> *char5 = new_node(14);
    Node<int> *char6 = new_node(15);
    Node<int> *char7 = new_node(16);

    // Connect the nodes
    char1->next = char2;
    char2->next = char3;
    char3->next = char4;
    char4->next = char5;
    char5->next = char6;
    char6->next = char7;

    binarySearchLL(char1, 18);

    return 0;
}
```

```
template <typename T>
Node<T> *getMiddle(Node<T> *start, Node<T> *last)
{
    Node<T> *slow = start;
    Node<T> *fast = start;

    while (fast != last && fast->next != last)
    {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

template <typename T>
void binarySearchLL(Node<T> *start, T dataFind)
{
    Node<T> *last = nullptr;

    while (start != last)
    {
        Node<T> *middle = getMiddle(start, last);

        if (middle->data == dataFind)
        {
            std::cout << "searching is Successfull\n";
            return;
        }
        else if (middle->data > dataFind)
        {
            last = middle;
        }
        else
        {
            start = middle->next;
        }
    }

    std::cout << "Searching is unsuccessful\n";
}
#endif
```

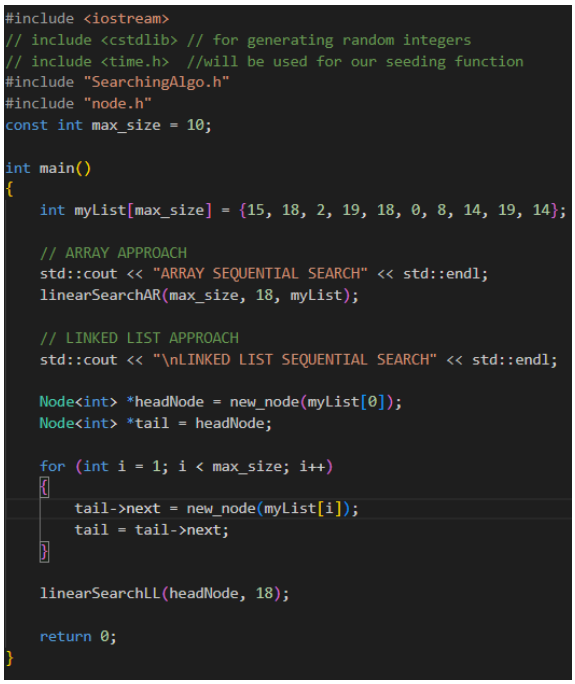
Screenshot

```
Searching is unsuccessful
```

Observations	The code first creates a sorted array containing the values 1 to 10 and then performs a binary search for 100. Since 100 is not included in the array, the array search is unsuccessful. After that, the code creates linked-list nodes containing 10 to 16 and connects them in order. It then uses binarySearchLL() to search for 18. Since 18 is not present in the linked list, the program displays “Searching is unsuccessful.”

Table 6-3b. Binary Search for Linked Listo

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1	
Screenshot	

```cpp
#include <iostream>
// include <cstdlib> // for generating random integers
// include <time.h>  //will be used for our seeding function
#include "SearchingAlgo.h"
#include "node.h"
const int max_size = 10;

int main()
{
    int myList[max_size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    // ARRAY APPROACH
    std::cout << "ARRAY SEQUENTIAL SEARCH" << std::endl;
    linearSearchAR(max_size, 18, myList);

    // LINKED LIST APPROACH
    std::cout << "\nLINKED LIST SEQUENTIAL SEARCH" << std::endl;

    Node<int> *headNode = new_node(myList[0]);
    Node<int> *tail = headNode;

    for (int i = 1; i < max_size; i++)
    {
        tail->next = new_node(myList[i]);
        tail = tail->next;
    }

    linearSearchLL(headNode, 18);

    return 0;
}
```

```cpp
template <typename T>
void linearSearchAR(int size, T dataFind, T array[])
{
    int i = 0;
    int comparisons = 0;

    while (i < size)
    {
        comparisons++;

        std::cout << "Comparisons made: "
                  << comparisons << "\n";

        if (dataFind == array[i])
        {
            std::cout << "Search successful\n";
            std::cout << "Total comparisons: "
                      << comparisons << "\n";
            return;
        }

        i++;
    }

    std::cout << "Search unsuccessful\n";
}
```

```cpp
template <typename T>
void linearSearchLL(Node<T> *nodes, T dataFind)
{
    int comparisons = 0;

    while (nodes != nullptr)
    {
        comparisons++;

        std::cout << "Comparisons made: "
                  << comparisons << "\n";

        if (nodes->data == dataFind)
        {
            std::cout << "Search successful\n";
            std::cout << "Total comparisons: "
                      << comparisons << "\n";
            return;
        }

        nodes = nodes->next;
    }

    std::cout << "Search unsuccessful\n";
}
```

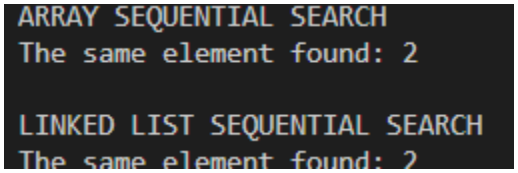
Output

```
c:\Users\mikaela\Downloads\GUARIN_CPE21S2\HOA6P1\output>.\"main.exe"
ARRAY SEQUENTIAL SEARCH
Comparisons made: 1
Comparisons made: 2
Search successful
Total comparisons: 2

LINKED LIST SEQUENTIAL SEARCH
Comparisons made: 1
Comparisons made: 2
Search successful
```

Observation

In this problem, I implemented a sequential search algorithm using both an array and a linked list to find the key value **18**. The program checks each element or node one by one starting from the beginning of the list until the value is found. In the array approach, the program compares the key with each index, while in the linked list approach, it traverses each node using the next pointer. Since the value **18** is located in the second position of the list, both approaches only require **two comparisons** before finding the value. This activity helped me understand that sequential search follows the same process whether the data is stored in an array or a linked list.

PROBLEM 2	
Screenshot	
Output	
Observation	In this problem, I modified the sequential search algorithm to count how many times the key value appears in the given list. Instead of stopping after finding the first match, the program continues checking all elements or nodes and increases the counter whenever the searched value is found. The given list contains two occurrences of 18, so the program returns a count of 2. Through this activity, I learned that sequential search can also be modified to find repeated values by allowing the program to continue searching until the end of the data.

PROBLEM 3

Screenshot

```
#include <iostream>
// include <cstdlib> // for generating random integers
// include <time.h>  //will be used for our seeding function
#include "SearchingAlgo.h"
#include "node.h"
const int max_size = 10;

int main()
{
    int myList[max_size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    binarySearchArr(myList, max_size, 8);

    return 0;
}
```

ITERATION 1:

[3,5,6,8,11,12,14,15,17,18]
0 1 2 3 4 5 6 7 8 9
↑ mid

11=8? NO
8<11
high = mid - 1
high = 3

ITERATION 2:

[3,5,6,8]
0 1 2 3
↑ mid = 1

5=8? NO
8<5
low = mid+1
low = 2

ITERATION 3:

[6,8]
↳ mid = 2

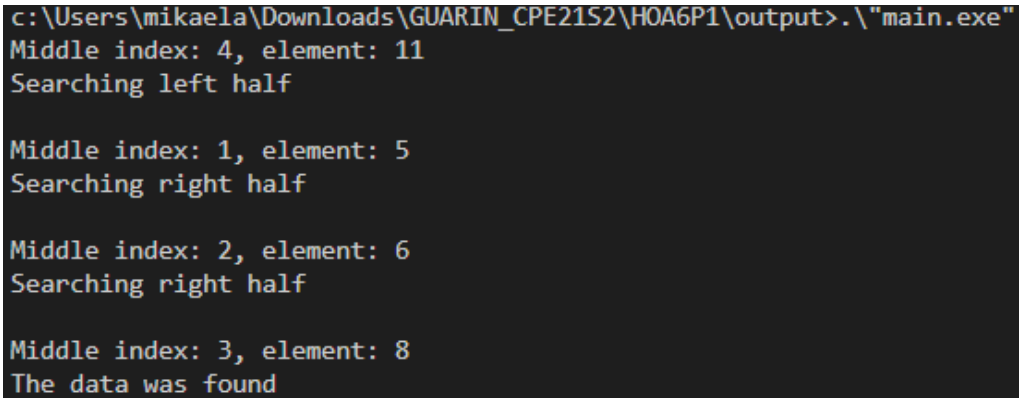
6=8? NO
8<6
low = mid+1
low = 3

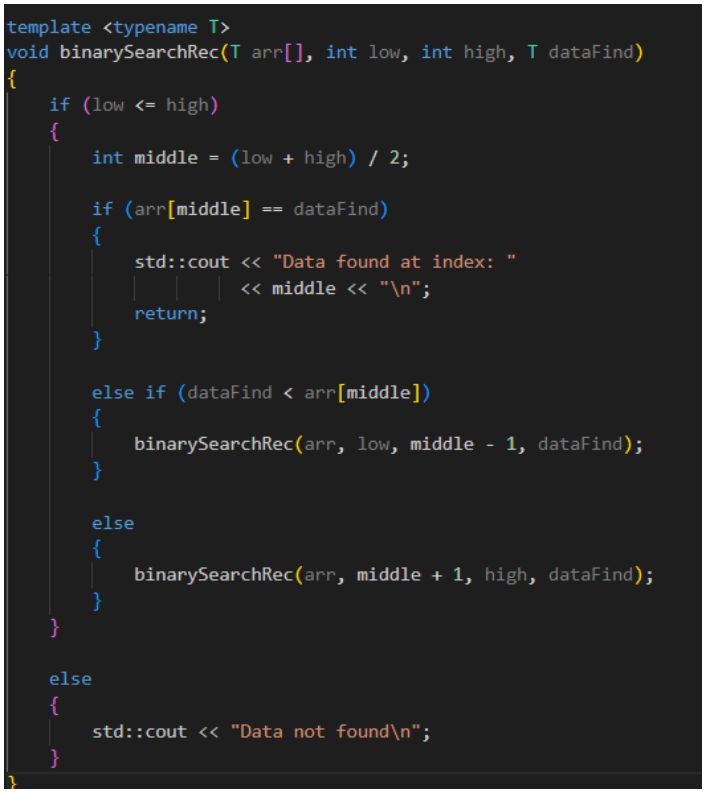
ITERATION 4:

[8]
↳ mid = 3

8=8? YES

Return

Output	
Observation	In this problem, I implemented a binary search algorithm to find the key value 8 from a sorted list. The program works by finding the middle element and comparing it with the search value. If the key is smaller than the middle element, it searches the left half; otherwise, it searches the right half. The process continues until the value is found. The output of the program matches the manual binary search steps, proving that the algorithm correctly reduces the search area and successfully finds the value.

PROBLEM 4	
Screenshot	

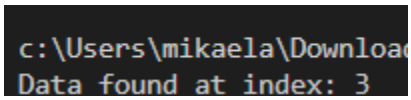
```
template <typename T>
void binarySearchRec(T arr[], int low, int high, T dataFind)
{
    if (low <= high)
    {
        int middle = (low + high) / 2;

        if (arr[middle] == dataFind)
        {
            std::cout << "Data found at index: "
                      << middle << "\n";
            return;
        }

        else if (dataFind < arr[middle])
        {
            binarySearchRec(arr, low, middle - 1, dataFind);
        }

        else
        {
            binarySearchRec(arr, middle + 1, high, dataFind);
        }
    }

    else
    {
        std::cout << "Data not found\n";
    }
}
```

Output	
Observation	In this problem, I modified the binary search algorithm into a recursive function. Instead of using a loop, the function repeatedly calls itself with a smaller search range depending on the comparison between the key value and the middle element. The program continues dividing the sorted list until the target value 8 is found or the search range becomes empty. This activity helped me understand how recursion can be applied in searching algorithms and how it can perform the same process as an iterative binary search.

CONCLUSION:
This activity helped me understand how different searching algorithms are implemented and how they work in solving problems. I learned that sequential search checks each element one by one, while binary search improves the searching process by repeatedly dividing a sorted list. I was also able to understand how recursion can be applied to binary search by allowing the function to call itself with a smaller search range. Through the exercises, I improved my understanding of arrays, linked lists, and the importance of choosing the proper searching technique depending on the data being used. I still need more practice in creating recursive functions and applying these algorithms to more complex problems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.